KingsHire (kingshire-v3) — Agent Handover Document

Date produced: 2026-06-26
Purpose: Complete project state handover for AI agents or developers picking up this codebase.

---

1. What This Project Is

KingsHire is a community-first freelance marketplace for members of Believer's Love World / Christ Embassy.

- **Live URL:** https://kingshire.uk
- **Staging URL:** Railway staging environment (auto-deploys from `staging` branch)
- **Two primary user roles:**
  - **Client** — Posts jobs, hires workers, pays via escrow
  - **Kinglancer** — The worker/freelancer who applies for and completes jobs
  - A user can hold both roles simultaneously
- **Admin panel:** `/admin` — protected by `ADMIN_PASSCODE` env var + HTTP-only cookie session

---

2. Tech Stack

Layer	Technology	Version
Framework	Next.js (App Router)	16.2.4
UI	React	19.2.4
Language	TypeScript	~5.x
Styling	Tailwind CSS	v4 (breaking: use `bg-linear-to-*` NOT `bg-gradient-to-*`)
Animations	Framer Motion	^12
Database	Supabase (PostgreSQL)	`@supabase/ssr` 0.10.3
Auth	Supabase Auth (PKCE) + KingsChat SSO + Google OAuth	—
Payments	Stripe Connect (separate charges/transfers)	—
Email	Brevo (formerly Sendinblue)	REST API via `BREVO_API_KEY`
Hosting	Railway (persistent server, NOT serverless/edge)	—
Icons	lucide-react	^1.8

Key architectural notes

- **NOT serverless.** Railway runs a persistent Node.js server. In-memory Next.js data cache **persists between requests but is cleared on every deploy/restart**.
- **`proxy.ts` is NOT Next.js middleware.** It is a separate Express proxy server for Railway routing.

- **`revalidateTag(tag, { expire: 0 })`** is valid in this Next.js version — `CacheLifeConfig = { expire?: number }`.
- **`unstable_cache`** is used for ISR-style caching.
- **`getDashboardContext`** is wrapped in React `cache()` for per-render deduplication within a single request.
- **Supabase keys:** Uses the new `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` format (not the old `NEXT_PUBLIC_SUPABASE_ANON_KEY`).

---

3. Supabase Database

Project

- **URL:** https://mdzousozzrnggtblusws.supabase.co
- **Project ref:** `mdzousozzrnggtblusws`
- **Service role key:** stored as `SUPABASE_SECRET_KEY` env var

Tables

Table	Purpose
`profiles`	One per auth user. Role, bio, services (JSONB), stripe info, rating, avatar, etc.
`jobs`	Job postings. Statuses: `open`, `in_progress`, `completed`, `cancelled`, `disputed`, `approved`.
`applications`	Kinglancer applications to open jobs.
`transactions`	Escrow records. One per job (`transactions_job_id_unique` unique constraint). Statuses: `held`, `released`, `refunded`, `transferred`.
`payment_attempts`	Tracks Stripe PaymentIntents before finalization. Replaces older "legacy" direct transactions.
`notifications`	In-app notifications. Read via `NotificationBell` component polling.
`reviews`	Double-blind review system (both parties submit; revealed simultaneously after both submit OR after 14-day timeout).
`disputes`	Disputes raised on `in_progress`/`completed` jobs.

Important DB constraints

- `transactions_job_id_unique` — only ONE transaction per job (prevents double-charging). This is intentional.
- RLS enabled on all tables.
- Service role client (`createServiceClient()` from `lib/supabase/service.ts`) bypasses RLS — used in all API routes and webhook handlers.

Migrations

Files in `supabase/migrations/` — numbered `002` through `026`. **These are NOT auto-applied.** Each must be run manually in the Supabase SQL Editor.

Migration 027 (`supabase/migrations/027_review_query_indexes.sql`) has been created but NOT yet run on staging or production. It adds two performance indexes:

```sql
create index if not exists idx_reviews_reviewer_job
  on public.reviews (reviewer_id, job_id);

create index if not exists idx_reviews_reviewee_published_at
  on public.reviews (reviewee_id, published_at desc)
  where is_published;
```

Action required: Run this SQL manually in Supabase SQL editor for both staging and production.

---

4. Environment Variables

All must be set in Railway service environment settings.

Variable	Required	Description
`NEXT_PUBLIC_SUPABASE_URL`	✅	Supabase project URL
`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`	✅	Supabase anon/publishable key (new format)
`SUPABASE_SECRET_KEY`	✅	Supabase service role key
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	✅	Stripe publishable key
`STRIPE_SECRET_KEY`	✅	Stripe secret key
`STRIPE_WEBHOOK_SECRET`	✅	Stripe webhook signing secret
`NEXT_PUBLIC_APP_URL`	✅	Full base URL (e.g. `https://kingshire.uk`)
`BREVO_API_KEY`	✅ for email	Brevo (email provider) API key
`BREVO_SENDER_EMAIL`	✅ for email	From address (e.g. `noreply@kingshire.uk`)
`BREVO_SENDER_NAME`	optional	Display name (e.g. `KingsHire`)
`ENABLE_EMAIL`	✅ for email	Must be `true` to send any emails
`ADMIN_PASSCODE`	✅	Password for admin panel login
`ADMIN_SESSION_SECRET`	✅	Secret for signing admin HTTP-only cookie
`ADMIN_NOTIFICATION_EMAIL`	optional	Admin email for dispute notifications
`CRON_SECRET`	✅	Bearer token for cron job endpoints
`KINGSCHAT_CLIENT_ID`	✅	KingsChat OAuth client ID
`KINGSCHAT_API_KEY`	✅	KingsChat API key
`APP_ENV`	optional	`staging` or `production`

---

5. Branch / Deployment State

Branch	Railway Service	Current State

`main`	Production (kingshire.uk)	At commit `7756485` — does NOT include email fixes or webhook idempotency fix
`staging`	Staging	At commit `458019a` — includes email fixes + webhook idempotency fix

Staging-only commits (not yet on `main`)

In order (oldest first):

1. `97c332b` — "Surface pending reviews as Action Centre items with countdown"
2. `523087a` — **fix: send email notifications when a job is posted**
3. `ef5fbee` — **fix: client application notification links directly to the job**
4. `458019a` — **fix: idempotent webhook handling for duplicate payment_intent.succeeded** ← URGENT

All 4 commits need to be merged to `main` for production. Commit `458019a` is the most urgent (active production bug).

Feature work NOT on any branch

The following features were built in a previous session, then the staging branch was hard-reset to match production. These changes exist only in the previous session's git history and need to be **re-applied**:

- Profile completeness gating (onboarding bio requirement, listing filters, home page filter, direct URL blocking)
- Job cancellation feature (grace period refunds for `in_progress` jobs)
- Kinglancer dashboard incomplete-profile banner
- `ConfirmModal` error prop

---

6. Platform Fees

- **Client pays:** budget + 2.5% platform fee
- **Kinglancer receives:** budget – 5% platform fee
- **Total platform take:** 7.5% of job budget
- Constants in `lib/stripe.ts`: `PLATFORM_FEE_RATE_CLIENT = 0.025`, `PLATFORM_FEE_RATE_KINGLANCER = 0.05`
- Auto-release: **5 working days** after work marked complete with no client response

---

7. Feature Status — Complete Inventory

✅ Authentication

- Email/password sign-up (multi-step: role → details → verify email)
- Google OAuth sign-in
- KingsChat SSO (cross-site POST → 303 redirect fix — see security notes)
- Forgot/reset password
- Email verification for non-Google users
- Onboarding page: role, bio (kinglancer required), services (at least one priced), phone, portfolio_url, cv_url

✅ Kinglancer Public Profile

- Route: `/kinglancers/[id]`
- **Completeness gate:** profiles are only visible if `bio.trim()` is non-empty AND at least one service has `rate > 0`. Incomplete profiles return 404 on direct URL access.
- Listing page (`/kinglancers`) also filters out incomplete profiles in JS.
- Home page "Top Kinglancers" component also filters out incomplete profiles.
- Kinglancer dashboard shows a red banner if profile is incomplete.

✅ Job Posting

- Clients can post public jobs or send direct requests to specific kinglancers.
- Categories, deadline, budget, rate type (fixed/hourly/daily).
- On post: bulk in-app notifications sent to top 50 matching kinglancers + email fan-out via `emailJobAlert`.
- For direct requests: only the invited kinglancer is notified.

✅ Job Application Flow

- Kinglancers apply to open jobs.
- Client receives in-app notification + email linking directly to `/dashboard/client/jobs/[jobId]`.
- Client can accept/reject applications.
- Accepted kinglancer receives "job awarded" notification.

✅ Direct Request Flow

- Client can invite a specific kinglancer.
- Kinglancer can accept/decline/counter-propose.
- On accept: status becomes `accepted_pending_payment`.
- Client pays → job goes `in_progress`.

✅ Payment Flow (Stripe Connect)

- Separate charges/transfers model.
- `payment_attempts` table tracks the PaymentIntent before finalization.
- On `payment_intent.succeeded` webhook: `finalizePaymentAttempt()` called.
  - Inserts a `transactions` row (status: `held`).
  - For applications: calls `selectApplicant()` to accept the winning application.
  - For direct requests: marks job `in_progress`.
- **Idempotency fix (staging only, needs to reach production):** if two concurrent webhook deliveries race, the second hits `transactions_job_id_unique` (23505). Now caught gracefully — returns 200 OK instead of 500. Stripe stops retrying.
- Express checkout (Apple Pay / Google Pay) — on `feat/express-checkout-wallets` branch, merged to `main`.

✅ Escrow / Release Flow

- Work submitted → client approves → `fireTransfer()` to kinglancer's Stripe Connect account.
- Auto-release cron: `/api/cron/auto-release` — runs via Railway cron service.
- `GRACE_PERIOD_MS = 2 * 60 * 60 * 1000` (2 hours) for in-progress job cancellation.

✅ Review System (Double-Blind)

- Both client and kinglancer submit reviews independently after job completion.
- Reviews are hidden until **both** submit, or after a 14-day timeout.
- Cron: `/api/cron/reveal-reviews` — reveals timed-out reviews.
- Pending reviews surface in the Action Centre with a countdown timer.

- Dashboard performance indexes in migration 025/026.

✅ Dispute System

- Client or kinglancer can raise a dispute on an `in_progress` or `completed` job.
- Admin receives notification.
- Admin resolves via `/admin/disputes`.

✅ Admin Panel

- Route: `/admin` (protected by passcode + HTTP-only cookie)
- Pages: `/admin/users`, `/admin/jobs`, `/admin/disputes`
- Pagination, search.

✅ Notification System

- In-app: `notifications` table, `NotificationBell` component polls for unread count.
- Email: via Brevo. Only fires when `ENABLE_EMAIL=true` AND `BREVO_API_KEY` AND `BREVO_SENDER_EMAIL` are set.
- Email helper functions in `lib/notifications.ts`:
  - `notify()` — creates in-app notification + optionally sends email
  - `emailJobAlert()` — email-only (no DB row), for fan-out on job post
  - `notifyNewApplication()` — client gets email with direct job link
  - `notifyJobAwarded()`, `notifyWorkSubmitted()`, `notifyPaymentReleased()`, `notifyDisputeRaised()`, `notifyPayoutReady()`, `notifyJobCancelled()`

✅ Job Cancellation

- Route: `POST /api/jobs/[id]/cancel`
- `open` **jobs:** bulk-rejects all applications + marks cancelled + notifies invited kinglancer (if direct request).
- `in_progress` **jobs:** checks transaction `created_at` within 2-hour grace period. If within: Stripe refund + update tx to `refunded` + cancel job + notify kinglancer. If outside: 409 `GRACE_PERIOD_EXPIRED`.
- UI: `CancelJobButton` client component on job workspace page.
- **Note:** This feature was built but is currently NOT on the staging or main branches (was reset). Needs to be re-applied.

✅ KingsChat SSO

- Cross-site POST from KingsChat hits `/auth/callback`.
- **Critical fix:** uses 303 redirect (not 307) so SameSite=Lax session cookies survive.
- **Security:** strictly requires `kcProfile.is_email_verified === true` before linking — prevents account takeover.
- Links KingsChat accounts to Supabase by email match.

✅ Stripe Connect Onboarding

- Kinglancers must complete Stripe Connect onboarding before receiving payouts.
- `stripe_onboarding_complete` boolean on profile.
- Webhook `account.updated` syncs payout status.
- After onboarding completes, automatically fires any pending unreleased transfers.

❌ Organisation Feature (PLANNED, NOT STARTED)

See Section 9 below.

---

8. Key File Locations

API Routes

Route	File	Description
`POST /api/jobs`	`app/api/jobs/route.ts`	Create job + fan-out notifications/emails
`POST /api/jobs/[id]/cancel`	`app/api/jobs/[id]/cancel/route.ts`	Cancel job with optional refund
`POST /api/applications`	`app/api/applications/route.ts`	Submit application
`POST /api/profile/complete-onboarding`	`app/api/profile/complete-onboarding/route.ts`	Save onboarding data
`POST /api/webhooks/stripe`	`app/api/webhooks/stripe/route.ts`	Stripe webhook handler
`GET /api/health`	`app/api/health/route.ts`	Health check
`GET /api/cron/auto-release`	`app/api/cron/auto-release/route.ts`	Auto-release escrow
`GET /api/cron/reveal-reviews`	`app/api/cron/reveal-reviews/route.ts`	Reveal timed-out reviews
`GET /api/cron/cleanup-abandoned-checkouts`	`app/api/cron/cleanup-abandoned-checkouts/route.ts`	Cancel stale PaymentIntents

Library Files

File	Description
`lib/notifications.ts`	All notification + email helpers
`lib/db/payment-attempts.ts`	PaymentAttempt DB operations + `finalizePaymentAttempt()`
`lib/db/applications.ts`	Application DB operations + `selectApplicant()`
`lib/db/jobs.ts`	Job DB operations
`lib/db/transactions.ts`	Transaction DB operations
`lib/stripe.ts`	Stripe helpers, fee constants, `fireTransfer()`
`lib/stripe-connect.ts`	Stripe Connect account creation
`lib/admin-auth.ts`	Admin passcode auth
`lib/roles.ts`	Role check helpers
`lib/pagination.ts`	Shared pagination utility
`lib/supabase/service.ts`	`createServiceClient()` — service role, bypasses RLS
`lib/supabase/server.ts`	`createServerClient()` — user session, respects RLS

`lib/supabase/client.ts`	Browser Supabase client

Dashboard Pages

Route	File
`/dashboard` (shell)	`app/(dashboard-shell)/layout.tsx`
Client dashboard	`app/(dashboard-shell)/dashboard/`
Client job workspace	`app/(dashboard-shell)/dashboard/client/jobs/[id]/page.tsx`
Kinglancer dashboard	`app/(dashboard-shell)/dashboard/kinglancer/page.tsx`
Profile edit	`app/(dashboard-shell)/dashboard/profile/page.tsx`

---

9. Planned Features (Next Steps)

IMMEDIATE — Production fix (URGENT)

Problem: The webhook idempotency fix (commit `458019a`) is on `staging` but NOT on `main` (production). Production is currently in a Stripe webhook retry loop every time a payment succeeds.

Action:

```
git checkout main
git merge staging --no-edit
git push origin main
```

Note: Before merging, confirm the staging-only email features are ready to ship to production (they should be safe — they only activate when `ENABLE_EMAIL=true`).

HIGH PRIORITY — Test email notifications on staging

What to test:

1. Set `ENABLE_EMAIL=true` in Railway staging environment (and redeploy if not done already).
2. Post a new job as a client → check that kinglancers receive an email.
3. Apply to a job → check that the client receives an email with a direct link to the job page (`/dashboard/client/jobs/[jobId]`).

Files involved: `lib/notifications.ts`, `app/api/jobs/route.ts`, `app/api/applications/route.ts`

HIGH PRIORITY — Run migration 027 on staging and production

Run this SQL manually in Supabase SQL Editor for **both** staging and production databases:

```
create index if not exists idx_reviews_reviewer_job
  on public.reviews (reviewer_id, job_id);

create index if not exists idx_reviews_reviewee_published_at
```

```
  on public.reviews (reviewee_id, published_at desc)
  where is_published;
```

File: `supabase/migrations/027_review_query_indexes.sql`

HIGH PRIORITY — Re-apply reset feature work to staging

The following features were built but then the staging branch was hard-reset (`git reset --hard 97c332b`) to match production. These need to be re-implemented:

1. Kinglancer Profile Completeness Gating

Rule: A kinglancer profile is "complete" if `bio` is non-empty AND at least one service in `services[]` has `rate > 0` .

Changes needed:

`app/onboarding/page.tsx`

- Add `bio` state (string, required for kinglancers, max 500 chars)
- Add "About you" textarea section before the services section (kinglancer-only)
- Validation: bio required + at least one service with `rate > 0`
- Pass `bio` in the API body

`app/api/profile/complete-onboarding/route.ts`

- Accept `bio` in request body
- Validate: bio not empty and <= 500 chars for kinglancers
- Validate: at least one service has `rate > 0` for kinglancers
- Include `bio` in the DB update for kinglancer role

`app/kinglancers/page.tsx`

- `getKinglancers()` must select `bio`
- Filter results: `filter(k => k.bio?.trim() && rawServices.some(s => Number(s.rate) > 0))`

`app/kinglancers/[id]/page.tsx`

- After `notFound()` check, add completeness check:

```
if (!kinglancer.bio?.trim() || !rawServices.some(s => Number(s.rate) > 0))
notFound();
```

`components/home/TopKinglancers.tsx`

- Fetch `bio` in select
- Filter out incomplete profiles with same rule
- Cache tag: `top-kinglancers` , 24h TTL

`app/(dashboard-shell)/dashboard/kinglancer/page.tsx`

- Compute `isProfileComplete = !!profile.bio?.trim() && services.some(s => Number(s.rate) > 0)`
- Show red banner when incomplete: `border-red-200 bg-red-50`

- Banner text: "Your profile is not visible to clients yet" + "Complete your profile →" link

2. Job Cancellation

New file: `app/api/jobs/[id]/cancel/route.ts`

```
POST /api/jobs/[id]/cancel
Auth: required, must be job owner (client_id)
```

Logic:

- If `status === 'open'` : bulk-reject applications + mark cancelled + notify invited kinglancer (direct request only)
- If `status === 'in_progress'` :
  - Get transaction → check `stripe_payment_intent_id` not null
  - Check `transaction.created_at` within `GRACE_PERIOD_MS = 2 * 60 * 60 * 1000` (2 hours)
  - If within grace: Stripe refund + update transaction to `refunded` + cancel job + `notifyJobCancelled()` to kinglancer
  - If outside grace: return 409 with `code: "GRACE_PERIOD_EXPIRED"`
- Other statuses: 400

New file: `app/(dashboard-shell)/dashboard/client/jobs/[id]/CancelJobButton.tsx`

- Client component
- Props: `jobId: string` , `status: "open" | "in_progress"` , `hasApplications?: boolean`
- Uses `ConfirmModal` with state-appropriate copy
- Redirects to `/dashboard/client/jobs` on success
- On grace period expired: shows inline error in modal

`app/(dashboard-shell)/dashboard/client/jobs/[id]/page.tsx`

- For `open` jobs: render `<CancelJobButton jobId={id} status="open" hasApplications={applications.length > 0} />` below applicants section
- For `in_progress` jobs: render `<CancelJobButton jobId={id} status="in_progress" />` in job workspace card

`components/ConfirmModal.tsx`

- Add optional `error?: string` prop
- Render red alert box inside modal above action buttons when `error` is set

---

DEFERRED — Organisation Feature

This is a planned new user type. Team message has been sent; waiting for responses.

Key agreed decisions so far:

- New role type named **"Organisation"** (not "Company", not "Business")
- Monthly subscription fee for access to the volunteer pool
- Verification against Companies House or Charity Commission register (admin manually certifies)
- Public organisation profile page
- **"Open to placements"** toggle on kinglancer profile (opt-in to volunteer work)
- Admin awards a certification badge to verified orgs (revocable)
- Disputes for volunteer placements go to admin

- 6-month maximum per placement
- Placement agreement auto-generated on both sides
- Experience portfolio on kinglancer profile (shows completed placements)

Not yet decided: specific subscription pricing, whether kinglancers can be paid by orgs at a reduced rate vs fully volunteer, exact UI for org discovery/search.

---

10. Known Bugs / Issues

Active Production Bug

- **Stripe webhook retry loop:** `payment_intent.succeeded` delivered twice concurrently → second delivery hits `transactions_job_id_unique` (23505) → webhook returns 500 → Stripe retries infinitely. **Fix is on staging (`458019a`), not yet on production.**
- **Impact:** Noisy logs, Stripe dashboard shows webhook failures. No double-charging occurs (constraint prevents it). The correct transaction record IS created.

Non-critical / Known

- Migration `027` not yet applied to any database — missing review query indexes. Queries still work, just slower at scale.
- Profile completeness feature was reset from staging — incomplete kinglancer profiles are currently visible on the listing and home page.
- Job cancellation feature was reset from staging — clients have no way to cancel jobs via UI.

---

11. Stripe Payment Flow — Detailed

```
1. Client selects kinglancer (application or direct request)
2. POST /api/payment-attempts/create
   → createPaymentAttempt() — inserts payment_attempts row (status: 'pending')
   → Stripe.paymentIntents.create() — creates PI
3. Client completes payment on /checkout page
4. Stripe fires payment_intent.succeeded webhook
5. POST /api/webhooks/stripe
   → finalizePaymentAttempt(pi.id)
     a. Looks up payment_attempts by stripe_payment_intent_id
     b. Checks for existing transaction on job_id (idempotency)
        - If exists + same PI: update to 'held', return finalizedNow: false
        - If exists + different PI: throw (different payment already funded job)
        - If 23505 on insert: catch gracefully, return finalizedNow: false
(IDEMPOTENCY FIX)
     c. Validates job state
     d. For applications: calls selectApplicant() to accept winner
     e. For direct requests: updates job to in_progress
     f. Inserts transaction (status: 'held')
     g. Updates payment_attempt to 'succeeded'
     h. Returns { attempt, finalizedNow: true }
   → Notifies kinglancer they were hired
6. Client approves work (or 5-day auto-release timer expires)
7. POST /api/jobs/[id]/approve or cron triggers
   → Update transaction to 'released'
```

```
   → fireTransfer() — Stripe transfer to kinglancer's Connect account
     (amount = budget * (1 - 0.05))
8. Stripe fires account.updated webhook (after Connect onboarding)
   → If any pending released transactions, fire queued transfers
```

---

12. Cron Jobs

Configured in `railway.toml` as separate Railway services.

Cron	Schedule	Route	Description
auto-release	Every hour	`/api/cron/auto-release`	Releases escrow after 5 working days
reveal-reviews	Daily	`/api/cron/reveal-reviews`	Reveals reviews after 14-day timeout
cleanup-abandoned-checkouts	Every 30 min	`/api/cron/cleanup-abandoned-checkouts`	Cancels stale PaymentIntents

All cron endpoints require `Authorization: Bearer ${CRON_SECRET}` header.

---

13. Security Notes

KingsChat SSO

- Callback uses **303 redirect** (not 307). 307 preserves the POST method, which blocks SameSite=Lax cookies. This fix is in `lib/kingschat-auth.ts`. Any future cross-site POST handlers must also use 303.
- **Email verification required:** `kcProfile.is_email_verified` must be `true` before linking accounts. Prevents account takeover where someone with an unverified KingsChat account could hijack a KingsHire account.

RLS

- All Supabase tables have Row Level Security enabled.
- Service role client (`createServiceClient()`) bypasses RLS — only used in server-side API routes, never exposed to client.
- Browser client uses `createBrowserClient()` which respects RLS.

Admin

- `/admin` routes protected by `ADMIN_PASSCODE` + HTTP-only cookie (signed with `ADMIN_SESSION_SECRET`).
- Admin routes in `app/admin/(protected)/` — layout checks cookie on every request.

---

14. How to Run Locally

```
npm install
npm run dev   # starts Next.js dev server on :3000
```

Required: `.env.local` with all env vars from Section 4.

For Stripe webhooks locally: `stripe listen --forward-to localhost:3000/api/webhooks/stripe`

---

15. Recommended Next Agent Prompt Prefix

When starting a new chat, paste this context:

> *This is the KingsHire (kingshire-v3) project. Next.js 16.2.4 App Router, Supabase, Stripe Connect, Brevo email, deployed on Railway. The AGENTS.md says "This is NOT the Next.js you know — read node_modules/next/dist/docs/ before writing code." Current branches: `staging` is ahead of `main` by 4 commits (webhook idempotency fix + email fixes). The most urgent task is merging staging to main to fix a production webhook retry loop. See `/docs/HANDOVER.md` in the repo for the full project state.*